Direct device access from the SmartNIC towards datacenter disagregation

Master's thesis meeting : week 11

Nicolas Jeanmenne

Table of contents

Poster

- Poster have been submitted (yay !)
- Kept in mind the note about IO-TCP optimization of TX and ZeroNIC RX for the poster presentation next Wednesday
- Thanks again for your useful reviews

Direct device access from the SmartNIC towards datacenter disagregation

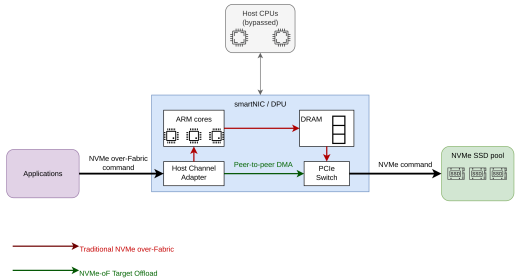
Author : Nicolas Jeanmenne Supervisors : Tom Barbette, Nikita Tyunyayev

Context

- End of Moore's Law and Dennard scaling has stalled CPU capacity while I/O device increases rapidly.
- CPU spend up to 70 % of cycles for I/O.
- Resource stranding lead from 10 % to 30 % wastes of server resources

Motivation

- Disaggregated hardware from the host to save CPU cycles.
- GPU and memory are heavily-studied
- SmartNIC-to-NVMe access needs further exploration.



Objectives

Modern solutions mainly lack of file synchronization [1] or write operations efficiency [2].

Our goal is to :

- Resolve file synchronization
- Maximize throughput, minimize latency in write operations

Approach

1. Direct storage access from smartNIC to SSDs
2. Direct access over the network
3. Hardware-accelerated encryption for secure transfer
4. Performance evaluation

State-of-the-art

	IO-TCP [1]	Scalio [2]	ZeroNIC [3]
Goal	Content delivery	Key-value store	Host networking
Zero copy	Yes	Yes	Yes
Large file delivery	Yes	No	Yes
Write latency	Yes	No	Yes
File synchronisation	No	No	Not stated

References

[1] T. Kim, D. M. Ng, J. Gong, Y. Kwon, M. Yu, and K. Park, "Rearchitecting the TCP Stack for I/O-Offloaded Content Delivery," presented at the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), 2023, pp. 275–292.Available: https://www.usenix.org/conference/nsdi23/presentation/kim-taehyun
[2] X. Sun, M. Zhang, Y. Shan, K. Chen, J. Jiang, and Y. Wu, "Scalio: Scaling up DPU-based JBOF Key-value Store with NVMe-oF Target Offload," presented at the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25), 2025, pp. 449–464. Available: https://www.usenix.org/conference/osdi25/presentation/sun
[3] A. Skiadopoulos et al., "High-throughput and Flexible Host Networking for Accelerated Computing," presented at the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), 2024, pp. 405–423. Available: https://www.usenix.org/conference/osdi24/presentation/skiadopoulos

About computational SSD

- More or less the same as accelerator + NVMe-oF
- Write latency improved in one of willow SSD app (*Caching*) because they use it as a custom cache

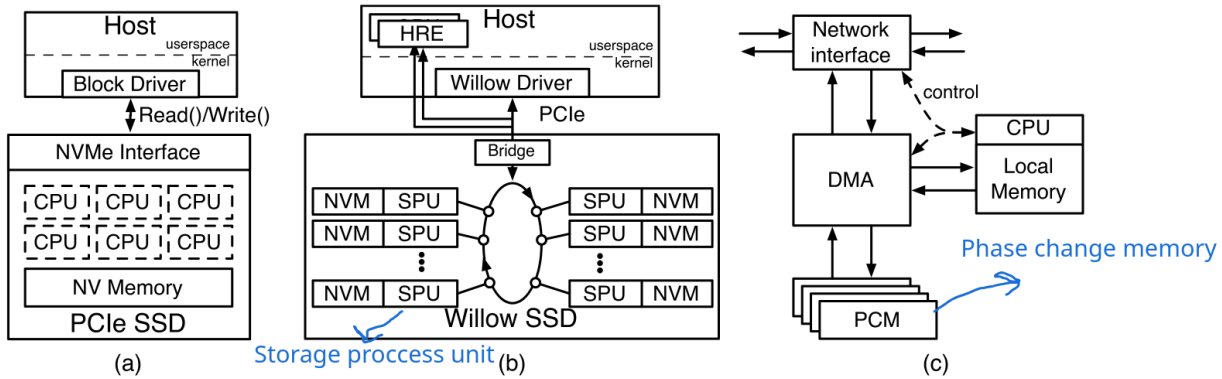


Figure 1: **A conventional SSD vs. Willow.** Although both a conventional SSD (a) and Willow (b) contain programmable components, Willow's computation resources (c) are visible to the programmer and provide a flexible programming model.

About computational SSD (cont.)

- Another SSD app (*Append*) about file system offload
  - If physical length > logical length, then special write issued and updated length logged
  - If not, host-side application invoke the file system
- Not directly linked with file synchronization of IO-TCP but could be a source of inspiration
- https://www.usenix.org/system/files/conference/osdi14/osdi14-paper-seshadri.pdf

Objectives / challenges

- IO-TCP
  - Lack of file synchronization
  - File-system change not propagated on NIC stack
- Scalio
  - Batched write efficient for throughput, not for latency
- ZeroNIC
  - Zero-copy limited by number of queues in the NIC

SOTA updates

- Rewrite problems section to architectural challenges
- Clear separation between high level disagregation problems and SOTA problems

3 Architectural challenges

Challenges whose origin is architectural design can be divided into two classes :

- Problems due to traditional monolithic architectural server. This type of problem is the core reason of why datacenters are moving from monolithic architecture from disaggregated one.

- New challenges brought by disaggregation. Disaggregated computing is a relatively new design in system and datacenter who breaks many of assumption previously made in the research. Researchers and datacenter operators need to find creative solutions to outcome these challenges and enjoy the benefits of disaggregation.

3.1 Monolithic challenges

The main drawback with monolithic architecture (MA) is **resource stranding**. When a server exhausts one of its resource, for example memory, all other resources cannot be utilized and result in a waste of resources.
Another non-negligible issue concerning MA is failure tolerance and availability. When a piece of hardware fails this may cause the entire server to fails and become unavailable.
Finally, replacing or upgrading hardware in MA occurs maintenance that can be costly.

3.2 Disaggregation challenges

One of the main challenge in disaggregated architecture (DA) is that network bandwidth becomes a central point of communication between clusters; CPU-storage communication needs to be done in a few milliseconds while CPU-memory needs to be done in a few nanoseconds, usually 100ns [1].
Existing network stacks face significant trade-offs in terms of flexibility and performance. For example, RDMA and RDMA over Convergent Ethernet (RoCE) are high-performance protocols, but they lack flexibility because they are tightly coupled to specific hardware, making them difficult to adapt. In addition to limited flexibility, RDMA suffers from issues like head-of-line blocking, congestion, and vendor lock-in, due to hardware requirements that aren't met by commodity equipment. On the other hand, network stacks like Linux TCP offer

SOTA updates

- SOTA design and solutions :
  - Added batched write section
  - Added Split stack section

4.2 Batched write

Batched write mechanism retains write operations until an arbitrary threshold is achieved, then all these operations are run consecutively while a group commit ensures atomicity of operations. The goal is to improve throughput and reduce the accelerator CPU load. While it is not technically a contribution brought by disaggregation, its evolution into such systems is still relevant to be mentioned.

4.3 Split stacks

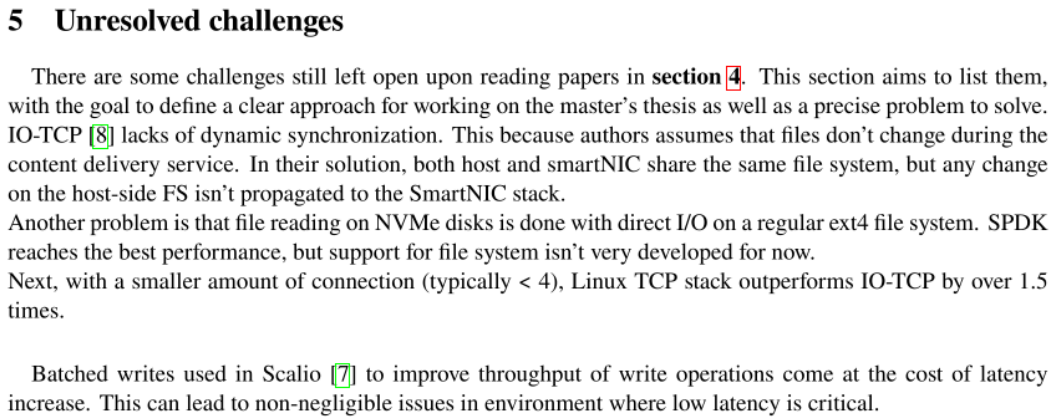
Split stacks are networking stacks divided in several planes, each one optimized for a specific task. IO-TCP from Kim et al. [8] separate the traditional Linux TCP stack into a stack running on the host CPU, performing control operations, and thus called the control plane. Such operations are complex and benefits from features of the CISC CPU (for example branch prediction, vectorized instructions, ...). The host stack extends the mTCP

3

[9] API with methods to offload file opening, writing and closing, such that applications have the most minimal changes to do in order to run itself on IO-TCP.
The data plane handles all operations related to data packet preparation and transfer. These operations are simpler in comparison of the control plane and then fit better the ARM cores of the SmartNIC. They also benefit from being stateless which suits the usage of the smartNIC. File and disk I/O are offloaded to otherwise it would interfere with the control plane on host CPU. Data plane operations are virtually performed and enable a mechanism of SEND / ECHO which translate in multiple MTU packets transfer.
Many difficulties due to the split introduce themselves, like RTT measurement, retransmission and error handling, but are essentially resolved with the set of commands provided by the implementation.

SOTA updates

- New section about unresolved challenges found in papers



5 Unresolved challenges

There are some challenges still left open upon reading papers in **section 4**. This section aims to list them, with the goal to define a clear approach for working on the master's thesis as well as a precise problem to solve.
IO-TCP [8] lacks of dynamic synchronization. This because authors assumes that files don't change during the content delivery service. In their solution, both host and smartNIC share the same file system, but any change on the host-side FS isn't propagated to the SmartNIC stack.
Another problem is that file reading on NVMe disks is done with direct I/O on a regular ext4 file system. SPDK reaches the best performance, but support for file system isn't very developed for now.
Next, with a smaller amount of connection (typically < 4), Linux TCP stack outperforms IO-TCP by over 1.5 times.

Batched writes used in Scalio [7] to improve throughput of write operations come at the cost of latency increase. This can lead to non-negligible issues in environment where low latency is critical.

Conclusion

- Poster is done, one deadline eliminated
- Focused on challenges yet to solve in SOTA

TODOs for week 13

- Complete unresolved challenges section in SOTA for other papers

That's all for today !